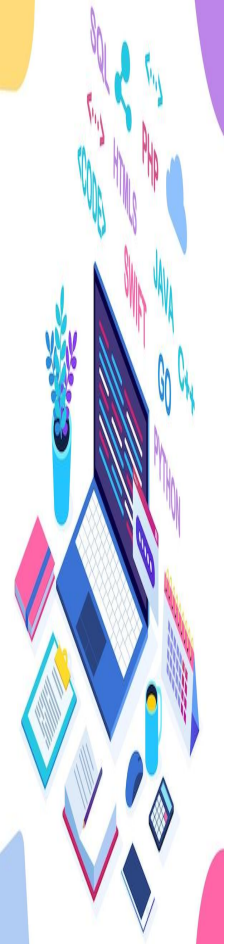


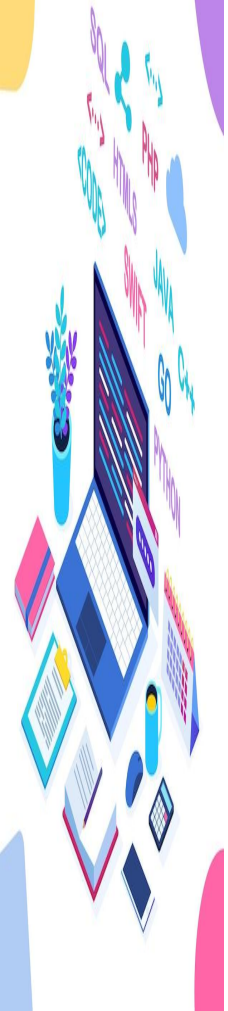
RAJALAKSHMI ENGINEERING COLLEGE

CS23333
Object Oriented Programming Using JAVA



Lambda Expression

- A lambda expression is essentially an anonymous (function without a name) method.
- It is used to implement a method defined by a functional interface.
- It provide a more compact syntax for implementing interface.
- A lambda expression in Java is a short, clear way to represent a single abstract method (SAM) from a functional interface — without writing a full class or method definition.

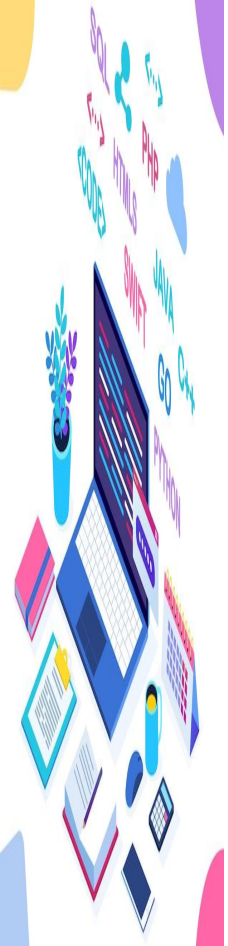


Lambda Expression

(parameters) -> expression

(parameters) -> { statements }

- **Parameters:** Inputs to the lambda (just like method parameters)
- **Arrow token (->):** Separates parameters from the body
- **Body**: The code executed when the lambda runs (can be one expression or a block {})



Functional Interface

- Examples:

1. No parameters

```
() -> System.out.println("Hello, Lambda!");
```

2. One parameter

```
(x) -> x * x
```

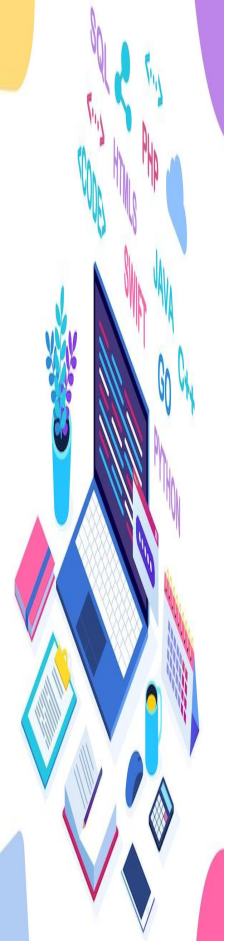
(This lambda returns the square of x)

3. Multiple parameters

```
(a, b) -> a + b
```

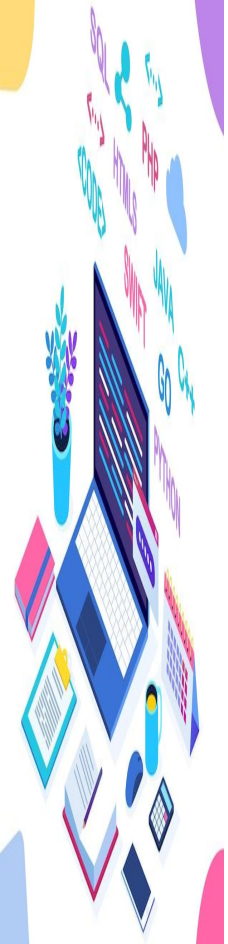
4. With a code block

```
(a, b) -> {
    int result = a + b;
   System.out.println("Sum = " + result);
}
```



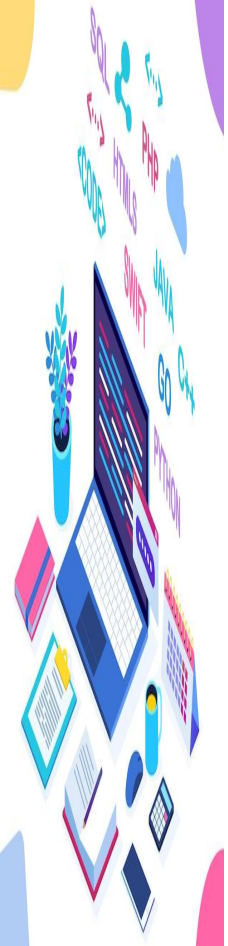
Functional Interface

- A functional interface is an interface with exactly one abstract method.
- This single abstract method specifies the intended purpose or action of the interface.
- **Example:** The standard interface Runnable is a functional interface.
- It defines only one method: run(). Therefore, the run() method defines the action of Runnable.
- A functional interface defines the target type of a lambda expression.
- A lambda expression can be used only in a context where its target type is known.
- A functional interface is sometimes called a SAM type, where SAM stands for Single Abstract Method.



Functional Interface

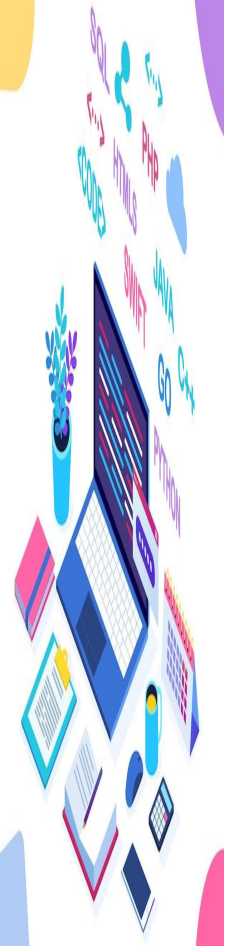
- Before Java 8:
- All interface methods were implicitly abstract (unless they were static).
- Since Java 8:
- You can define default, static, and even private methods inside interfaces — these methods have their own implementations.
- Therefore, an interface method is abstract only if it does not have an implementation.
- So, a method in an interface is abstract when it:
    - **Is not default**
    - **Is not static**
    - **Is not private**
    - **And does not have a body {}**
- That's why you don't need to write the abstract keyword anymore — it's implied.



Functional Interface

```
interface MyNumber {
    double getValue();
}
public class OneParameterExample {
    public static void main(String[] args) {
// MyNumber myNum = () -> 123.45;
MyNumber myNum = new MyNumber() {
      public double getValue() {
        return 123.45;
    }
};
System.out.println(myNum.getValue());
}
}
```





Functional Interface

Lambda Expression Without Parameters

- **Define the Functional Interface**

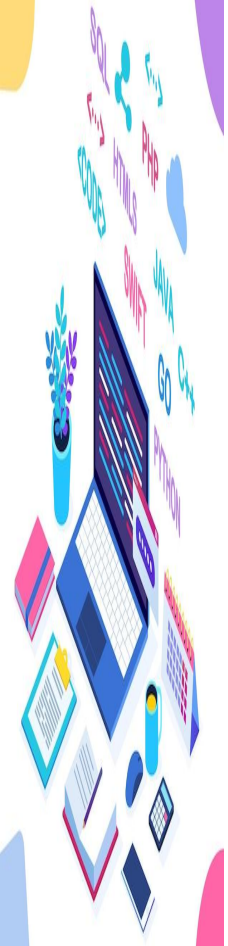
```
interface MyNumber {
    double getValue();
}
```

  - getValue() is abstract (no implementation).
  - It's the only abstract method → functional interface.

- **Declare a Reference**

```
MyNumber myNum;
```

  - This line just declares a reference to an object that implements MyNumber.
  - No object exists yet — it's just a variable that can hold such an object.



Functional Interface

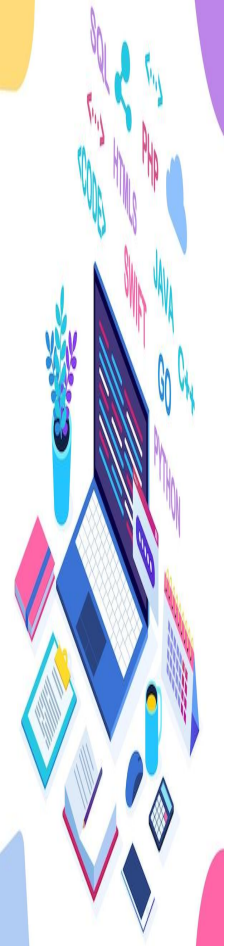
- **Assign a Lambda Expression**

```
myNum = () -> 123.45;
```

  - Here:
    - The target type of the lambda is MyNumber.
    - The compiler sees that MyNumber has one method: double getValue().
    - So it creates an anonymous class behind the scenes that implements getValue() and returns 123.45.

- Effectively, this is similar to:

```
MyNumber myNum = new MyNumber() {
    public double getValue() {
      return 123.45;
    }
};
```



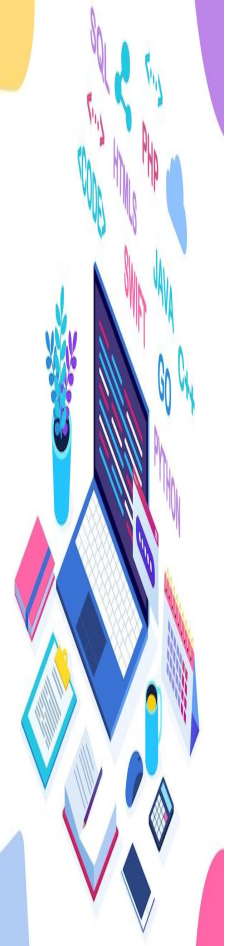
Functional Interface

- **Call the Method**

  System.out.println(myNum.getValue());

- Output:
- 123.45

  - When getValue() is called:
  - The lambda expression () -> 123.45 is executed.
  - It returns 123.45



Functional Interface

Lambda Expression With One Parameter

```
interface NumericTest {
    boolean test(int n);
}
```

- Lambda Expression:

```
public class OneParameterExample {
    public static void main(String[] args) {
        // Lambda with one parameter
        NumericTest isEven = n -> n % 2 == 0;
         System.out.println(isEven.test(10)); // true
        System.out.println(isEven.test(7));  // false
    }
}
```

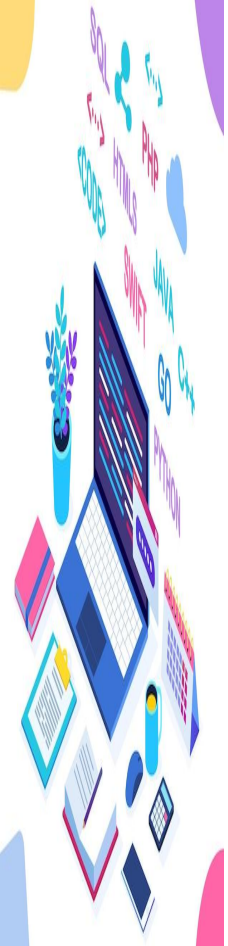
Output:

true

false

Explanation:

- The lambda n -> n % 2 == 0 takes one parameter (n).
- Parentheses are optional when there is only one parameter.
- It implements boolean test(int n).



Functional Interface

Lambda Expression With Two Parameters

```
interface MathOperation {
    int operate(int a, int b);
}
public class TwoParameterExample {
    public static void main(String[] args) {
        // Lambda with two parameters
        MathOperation add = (a, b) -> a + b;
        MathOperation multiply = (a, b) -> a * b;
        System.out.println("Addition: " + add.operate(5, 3));
        System.out.println("Multiplication: " +
multiply.operate(5, 3));
    }
}
```

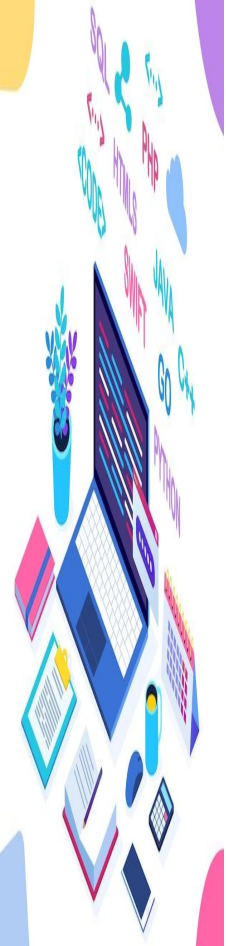
Output:

Addition: 8

Multiplication: 15

Explanation:

- The lambdas have two parameters: (a, b) -> ....
- Parentheses are required when there are two or more parameters.
- The body is a single expression that returns a value.



Functional Interface

Lambda Expression With a Block of Code (Multiple Statements)

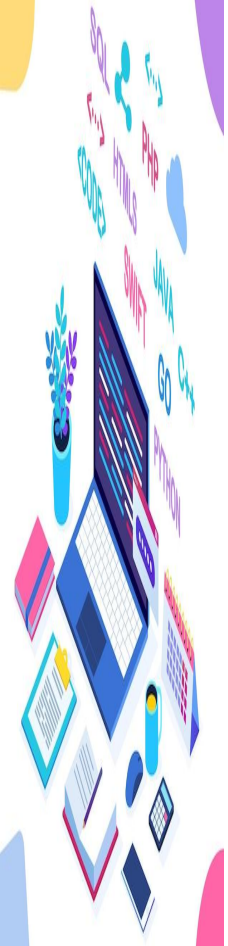
```
interface StringProcessor {
    String process(String str);
}
public class BlockLambdaExample {
    public static void main(String[] args) {
        // Lambda with a block of code
        StringProcessor reverse = (str) -> {
            String result = "";
            for (int i = str.length() - 1; i >= 0; i--) {
                result += str.charAt(i);
            }
            return result; // must use 'return' when using block
        };
        System.out.println(reverse.process("Lambda"));
    }}
```

Output:

adbmaL

Explanation:

- The lambda uses multiple statements, so it must be enclosed in { }.
- When using a block body, you must explicitly use the return keyword if the method returns a val



Predefined Functional Interface

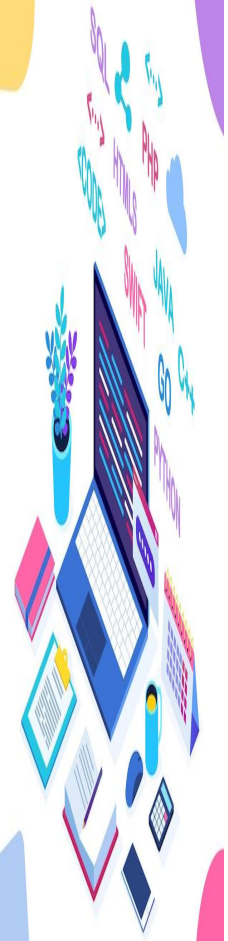
Example:

```
interface Calculator {
    int add(int a, int b);
}
public class Test {
    public static void main(String[] args) {
        Calculator c = (x, y) -> x + y;
        System.out.println(c.add(5, 3));
    }
}
```



Predefined Functional Interface

Interface	Package	Abstract Method	Description
Function<T, R>	java.util.function	R apply(T t)	Takes one argument, returns a result
Predicate	java.util.function	boolean test(T t)	Tests a condition and returns boolean
Consumer	java.util.function	void accept(T t)	Performs an action (no return)
Supplier	java.util.function	T get()	Supplies a value (no input)
BiFunction<T, U, R>	java.util.function	R apply(T t, U u)	Takes two arguments, returns a result
BiPredicate<T, U>	java.util.function	boolean test(T t, U u)	Tests condition with two arguments
BiConsumer<T, U>	java.util.function	void accept(T t, U u)	Performs an operation on two arguments
UnaryOperator	java.util.function	T apply(T t)	A Function where input and output are same type
BinaryOperator	java.util.function	T apply(T t1, T t2)	A BiFunction with same input/output type



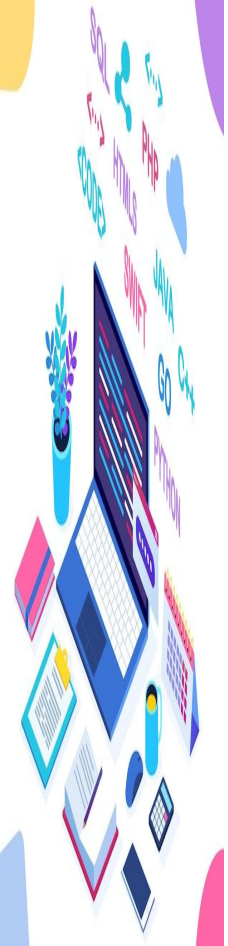
Predefined Functional Interface

```
import java.util.function.Function;

public class FunctionExample {
    public static void main(String[] args) {
        // Example: Convert String to Integer (length of string)
        Function<String, Integer> stringLength = str -> str.length();

        String name = "ChatGPT";
        int length = stringLength.apply(name);

        System.out.println("The length of \"" + name + "\" is: " + length);
    }
}
```

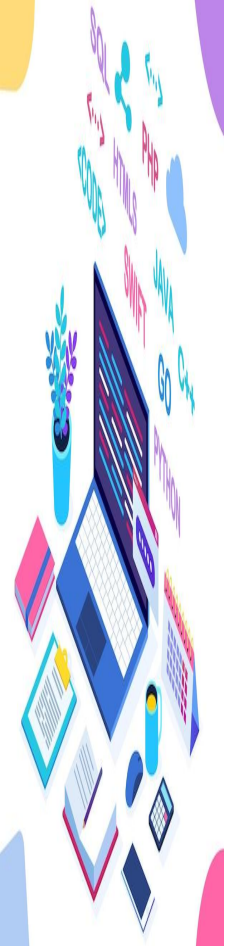


Predefined Functional Interface

```
import java.util.function.Predicate;

public class PredicateExample {
    public static void main(String[] args) {
        // Example: Check if a number is even
        Predicate<Integer> isEven = num -> num % 2 == 0;

        int number = 10;
        System.out.println(number + " is even? " + isEven.test(number));
    }
}
```

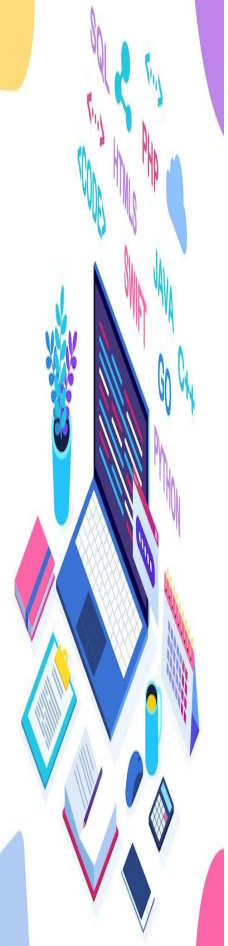


Predefined Functional Interface

```
import java.util.function.Consumer;

public class ConsumerExample {
    public static void main(String[] args) {
        // Example: Print a message
        Consumer<String> printMessage = message -> System.out.println("Message: "
+ message);

        printMessage.accept("Hello from Consumer!");
    }
}
```



Predefined Functional Interface

BiFunction<T, U, R>

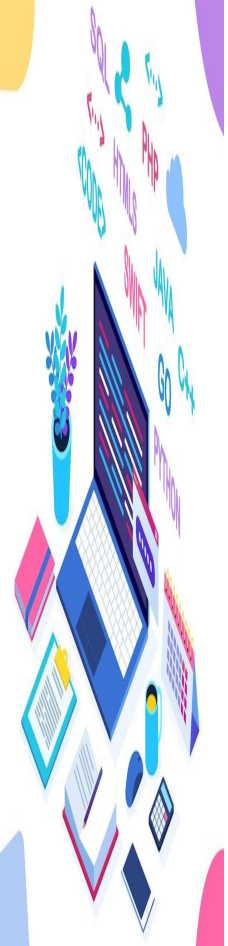
```
BiFunction<Integer, Integer, Integer> add = (a, b) -> a + b;
System.out.println(add.apply(5, 3)); // Output: 8
```

BiPredicate<T, U>

```
BiPredicate<String, Integer> longerThan = (str, len) -> str.length() >= len;
System.out.println(longerThan.test("Java", 3)); // true
```

BiConsumer<T, U>

```
BiConsumer<String, Integer> printDetails = (name, age) ->
    System.out.println(name + " is " + age + " years old");
printDetails.accept("Alice", 25); // Output: Alice is 25 years old
```



Predefined Functional Interface

UnaryOperator<T>

```
UnaryOperator<Integer> square = x -> x * x;
System.out.println(square.apply(5)); // Output: 25
```

BinaryOperator<T>

```
BinaryOperator<Integer> multiply = (a, b) -> a * b;
System.out.println(multiply.apply(4, 6)); // Output: 24
```